

RELATÓRIO DE AUDITORIA DE SEGURANÇA DE CÓDIGO

Revisão Arquitetural em 5 Categorias Críticas de Segurança SaaS / Web

Aplicação Alvo:	Plataforma Web EasyTraining (Next.js 16.3 / React 19 / Firestore)
Ambiente & Stack:	TypeScript 5.8, Tailwind CSS v4, Firebase Admin, Vercel Serverless
Data de Emissão:	14 de Setembro de 2026
Escopo da Revisão:	1. Banco sem Tranca 2. Permissão no Navegador 3. IDOR 4. Chaves Expostas 5. Inputs / XSS
Responsável Técnico:	Antigravity AI AppSec Core — Auditoria Automatizada & Estática
Classificação:	CONFIDENCIAL — PROPRIEDADE INTERNA
Status Geral:	9 Vulnerabilidades Mapeadas (3 Altas, 3 Médias, 3 Baixas)

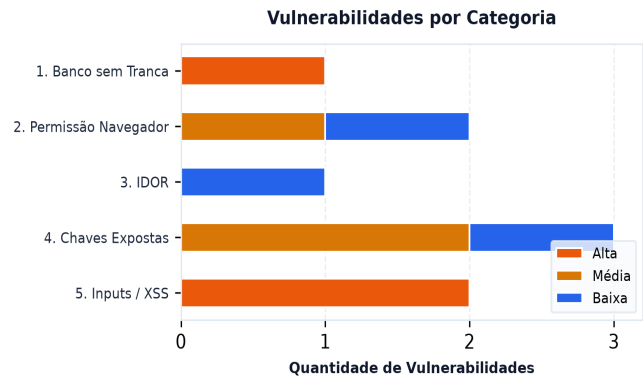
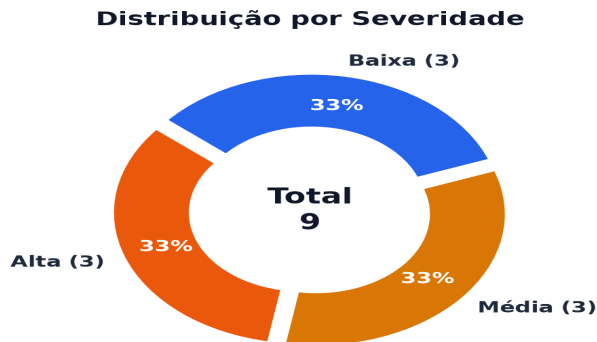
SUMÁRIO DO VEREDITO EXECUTIVO:

A auditoria de código fonte identificou sólidos mecanismos de segurança nos fluxos centrais do back-end, notadamente na proteção rígida dos leads no Firestore via regras de segurança atômicas (LGPD-compliant), validação estrita de assinaturas HMAC-SHA256 para sessões administrativas e checagem binária de magic numbers contra uploads de arquivos maliciosos.

Entretanto, foram identificadas **3 vulnerabilidades de severidade ALTA** que exigem remediação prioritária: vazamento público do webhook de automação (n8n), bypass de sanitização HTML via regex com renderização no blog (dangerouslySetInnerHTML), e interpolação direta de parâmetros de configuração em scripts inline (Stored XSS).

1. Resumo Executivo & Painel de Indicadores

A análise cobriu 21 rotas de API em src/app/api/, camadas de middleware, serviços de autenticação, regras do Firestore e renderizadores de interface. O resultado quantitativo consolidado está representado abaixo:



Matriz Geral de Vulnerabilidades Encontradas

ID	Categoria	Severidade	Arquivo e Linha	Impacto Resumido
C1-01	1. Banco sem Tranca	ALTA	firestore.rules:13-16 api/site-config:6-14	Exposição pública de URL privada do webhook n8n (CRM/Telegram)
C2-01	2. Permissão Navegador	MÉDIA	authService.ts:145-167	Dessincronização de sessão client/server no login Firebase Auth
C2-02	2. Permissão Navegador	BAIXA	src/middleware.ts:109	Rotas /api isentas do middleware global de segurança
C3-01	3. IDOR	BAIXA	api/posts/[id]:13 api/courses/[id]:13	Falta de validação de formato do ID dinâmico da rota
C4-01	4. Chaves Expostas	MÉDIA	authService.ts:21-26	Lista de e-mails administrativos exposta no bundle público client
C4-02	4. Chaves Expostas	MÉDIA	api/admin/login:37	Comparação de senha não segura contra Timing Attacks
C4-03	4. Chaves Expostas	BAIXA	Git commit a3b21d2	Credenciais de desenvolvimento em histórico antigo do Git
C5-01	5. Inputs / XSS	ALTA	lib/security.ts:41-58 PostDetailView.tsx:141	Bypass de regex em rich text com dangerouslySetInnerHTML
C5-02	5. Inputs / XSS	ALTA	src/app/layout.tsx:261, 281	Stored XSS via injeção de scripts inline GA4/GTM não validados

2. Pontos Fortes & Controles Verificados

A auditoria verificou meticulosamente onde a aplicação já implementou padrões adequados de segurança defensiva em todas as 5 categorias analisadas:

PONTO FORTE — 1. Banco sem Tranca (Isolamento de Dados)

Verificado em: firestore.rules:20-24

A coleção crítica `/leads/{leadId}` possui bloqueio estrito contra leitura, alteração ou exclusão por clientes (`allow read, update, delete: if false;`). A criação pública exige formato string para nome e telefone. Nenhum visitante consegue enumerar dados de alunos cadastrados diretamente no Firestore.

PONTO FORTE — 2. Permissão no Navegador (Server Guards)

Verificado em: Todas as 16 rotas de escrita em `src/app/api/`

Todas as rotas de mutação administrativas (criação/edição de cursos, posts, gerenciamento de leads, IA, sincronização) executam verificação server-side obrigatória via `verifyAdminSession(request)` antes de qualquer operação no banco.

PONTO FORTE — 3. IDOR (Isolamento Horizontal)

Verificado em: Arquitetura de Leads em `src/lib/leadsDb.ts`

A plataforma opera em modelo single-tenant para gestão institucional. Não há exposição de endpoint público `GET /api/leads/[id]`. Alunos não possuem credenciais de acesso, eliminando vetores de vazamento horizontal entre estudantes.

PONTO FORTE — 4. Chaves & Segredos Criptográficos

Verificado em: `src/lib/authServer.ts:4-10`

A função `getAdminSessionSecret()` força que a chave de assinatura de sessão possua no mínimo 32 caracteres, interrompendo a execução com erro caso o ambiente esteja mal configurado. A chave privada do Firebase Admin normaliza escape de quebras de linha PEM sem armazenar segredos de fallback hardcoded no código ativo.

PONTO FORTE — 5. Inputs sem Tratamento (Prevenção de SVG XSS)

Verificado em: `src/app/api/upload/route.ts:9-14`

O upload de mídia rejeita categoricamente arquivos com extensão ou MIME type `image/svg+xml`, e valida magic numbers binários para formatos WebP, PNG, JPEG e GIF, impedindo a injeção de vetores SVG Stored XSS na raiz.

3. Detalhamento Técnico das Vulnerabilidades

Abaixo estão documentadas as falhas identificadas, contendo arquivo e linha exatos, trecho vulnerável, vetor de exploração e o patch pronto para aplicação.

CATEGORIA 1: BANCO SEM TRANCA (Isolamento de Inquilino/Dono)

C1-01: Vazamento Público de n8nWebhookUrl via Firestore e API Aberta

ALTA

Localização: firestore.rules:13-16 e src/app/api/site-config/route.ts:6-14

Vetor de Ataque: A regra do Firestore concede leitura irrestrita da coleção config para qualquer cliente sem autenticação. Concomitantemente, o endpoint GET /api/site-config retorna o objeto com n8nWebhookUrl sem filtragem. Isso expõe a URL privada do webhook com tokens de autenticação do n8n/Telegram/CRM.

Código Vulnerável Atual:

```
// firestore.rules:13-16
match /config/{configId} {
  allow read: if true;
  allow write: if false;
}
```

Correção Recomendada (Pronta para Aplicação):

```
// src/app/api/site-config/route.ts
export async function GET() {
  const config = await getSiteConfigFromFirestore();
  const { n8nWebhookUrl, ...publicConfig } = config as any;
  return NextResponse.json(publicConfig);
}

// firestore.rules: Fechar leitura direta do cliente
match /config/{configId} {
  allow read, write: if false;
}
```

CATEGORIA 2: PERMISSÃO DEFINIDA NO NAVEGADOR

C2-01: Dessincronização de Sessão e Bypass Cosmético via LocalStorage no Firebase Auth

MÉDIA

Localização: src/services/authService.ts:145-167

Vetor de Ataque: No fallback via signInWithEmailAndPassword, o cliente salva o estado de autenticação no localStorage, mas não solicita a emissão do cookie HttpOnly admin_session assinado. A UI do painel abre, porém todas as chamadas às APIs administrativas falham com 401. Permite também forjar acesso visual via DevTools.

Código Vulnerável Atual:

```
// authService.ts:145-167
const userCredential = await signInWithEmailAndPassword(auth, normalizedEmail, pass);
// Salva no localStorage mas NÃO emite cookie no servidor
localStorage.setItem(ADMIN_STORAGE_KEY, JSON.stringify(sessionData));
```

Correção Recomendada (Pronta para Aplicação):

```
// Sincronizar sessão com a rota server-side para emissão do cookie HttpOnly:
const res = await fetch('/api/admin/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email: normalizedEmail, idToken: await fbUser.getIdToken() })
});
```

C2-02: Isenção Total das Rotas /api no Matcher de Segurança do Middleware**BAIXA****Localização:** src/middleware.ts:109

Vetor de Ataque: O regex matcher do middleware ignora todas as rotas com prefixo api (?!api...). Apesar de cada handler atual conter verifyAdminSession, a falta de verificação perimetral remove a camada de Defesa em Profundidade.

Código Vulnerável Atual:

```
// src/middleware.ts:109
matcher: ['/(?!(?!api|_next/static|_next/image|favicon.ico|manifest.json).*)'],
```

Correção Recomendada (Pronta para Aplicação):

```
// Adicionar guard perimetral no middleware para rotas de escrita administrativas:
if (request.nextUrl.pathname.startsWith('/api/admin/') &&
    !request.nextUrl.pathname.startsWith('/api/admin/login')) {
    const session = request.cookies.get('admin_session')?.value;
    if (!session) return NextResponse.json({ error: 'Não autorizado' }, { status: 401 });
}
```

CATEGORIA 3: IDOR (Insecure Direct Object Reference)**C3-01: Ausência de Validação de Formato de ID nas Rotas de Posts e Cursos****BAIXA****Localização:** src/app/api/posts/[id]/route.ts:13 e src/app/api/courses/[id]/route.ts:13

Vetor de Ataque: A rota de leads sanitiza estritamente os IDs (/^[a-zA-Z0-9_-]{3,64}\$/), enquanto posts e courses comparam strings brutas. Parâmetros malformados ou caracteres de controle podem gerar comportamentos inesperados.

Código Vulnerável Atual:

```
// src/app/api/posts/[id]/route.ts:13
const { id } = await params;
const post = posts.find(p => String(p.id) === String(id));
```

Correção Recomendada (Pronta para Aplicação):

```
// Validar formato estrito do ID:
function isValidEntityId(id: string): boolean {
    return /^[a-zA-Z0-9_-]{2,80}$/.test(id);
}
if (!isValidEntityId(id)) return NextResponse.json({ error: 'ID inválido' }, { status: 400 });
```

CATEGORIA 4: CHAVES EXPOSTAS (Hardcode & Defaults)

C4-01: Exposição da Lista de E-mails de Administradores no Bundle Público do Cliente

MÉDIA

Localização: src/services/authService.ts:21-26

Vetor de Ataque: A lista ALLOWED_ADMIN_EMAILS é empacotada no JavaScript público do navegador, permitindo que atacantes conheçam as contas privilegiadas da instituição e executem ataques de spear-phishing e enumeração.

Código Vulnerável Atual:

```
// src/services/authService.ts:21-26
export const ALLOWED_ADMIN_EMAILS = [
  'raccorreia@gmail.com',
  'rac2digital@gmail.com',
  'admin@easytraining.com.br',
  'easytraining.cursos@gmail.com'
];
```

Correção Recomendada (Pronta para Aplicação):

```
// Manter e-mails exclusivamente no back-end (authServer.ts):
// O cliente apenas submete as credenciais para validação no servidor.
```

C4-02: Comparação de Senha Mestra Vulnerável a Timing Attack no Endpoint de Login

MÉDIA

Localização: src/app/api/admin/login/route.ts:37

Vetor de Ataque: A comparação de strings com === encerra no primeiro caractere divergente, abrindo margem para inferência da senha de administração via medição estatística de tempo de resposta da API.

Código Vulnerável Atual:

```
// src/app/api/admin/login/route.ts:37
const isMasterValid = password === masterPassword;
```

Correção Recomendada (Pronta para Aplicação):

```
// Utilizar comparação em tempo constante com timingSafeEqual:
import { timingSafeEqual } from 'crypto';
const passBuf = Buffer.from(password.padEnd(64, ' '));
const masterBuf = Buffer.from(masterPassword.padEnd(64, ' '));
const isMasterValid = password.length === masterPassword.length &&
  timingSafeEqual(passBuf, masterBuf);
```

C4-03: Resíduos de Credenciais Antigas de Desenvolvimento em Histórico do Git

BAIXA

Localização: Git commit a3b21d2bb036b28327382fdeda30ad9e5cd5407e

Vetor de Ataque: Commits anteriores registraram senhas de teste como admin123 e easytraining2026. Em caso de abertura do código ou vazamento do repositório, essas strings podem ser reutilizadas.

Código Vulnerável Atual:

```
// Commit a3b21d2b...
- SENHA_PADRAO = 'admin123' / 'easytraining2026'
```

Correção Recomendada (Pronta para Aplicação):

```
// Ação de Higiene: Rotacionar todas as senhas de produção e higienizar histórico Git se o repositório for público.
```

CATEGORIA 5: INPUTS SEM TRATAMENTO (O Clássico Esquecido)

C5-01: Sanitização HTML Frágil por Regex e Stored XSS via dangerouslySetInnerHTML**ALTA****Localização:** src/lib/security.ts:41-58 e src/components/blog/PostDetailView.tsx:141**Vetor de Ataque:** A sanitização por expressões regulares falha contra tags como iframe, embed, object e manipuladores de evento sem espaçamento tradicional (ex: <svg/onload=alert(1)>). O HTML é renderizado diretamente no navegador do leitor.**Código Vulnerável Atual:**

```
// src/lib/security.ts:41-58
return input.replace(/<script\b[^\>]*(?:?!<\script>)<[^\>]*<\script>/gi, '')
  .replace(/\\son\\w+\\s*=\\s*("[\"]).*?\\1/gi, '') ...
```

Correção Recomendada (Pronta para Aplicação):

```
// Sanitização padrão indústria com isomorphic-dompurify:
import DOMPurify from 'isomorphic-dompurify';
export function sanitizeHtmlContent(input: unknown): string {
  if (typeof input !== 'string') return '';
  return DOMPurify.sanitize(input, {
    ALLOWED_TAGS: ['p', 'b', 'i', 'em', 'strong', 'a', 'h2', 'h3', 'h4', 'ul', 'ol', 'li', 'img', 'blockquote'],
    ALLOWED_ATTR: ['href', 'src', 'alt', 'title', 'class', 'target']
  });
}
```

C5-02: Stored XSS via Interpolação de Strings Não Sanitizadas em Scripts Inline de GA4/GTM**ALTA****Localização:** src/app/layout.tsx:261, 281**Vetor de Ataque:** Os identificadores gald e gtmId são inseridos sem validação alfanumérica dentro de tags <script> inline. Um valor hostil salvo na configuração injeta código JavaScript arbitrário em todas as páginas públicas e privadas.**Código Vulnerável Atual:**

```
// src/app/layout.tsx:261, 281
gtag('config', `${gaId}`, { ... });
})(window, document, 'script', 'dataLayer', `${gtmId}');
```

Correção Recomendada (Pronta para Aplicação):

```
// Validar estritamente o formato alfanumérico antes da injeção no DOM:
const GA_REGEX = /^[A-Z0-9-]{4,20}$/i;
const safeGaId = GA_REGEX.test(gaId || '') ? gaId : null;
const safeGtmId = GA_REGEX.test(gtmId || '') ? gtmId : null;
// Injetar script somente se safeGaId / safeGtmId forem válidos.
```

4. Plano de Remediação Priorizado

A remediação está dividida em 3 fases cronológicas de acordo com a criticidade do risco e impacto:

Fase / Prazo	ID	Ação Recomendada	Alvo / Arquivo	Esforço
P1 (Alta) Prazo: 24h	C1-01	Expurgar n8nWebhookUrl do GET /api/site-config e bloquear leitura pública de /config/* no Firestore	firestore.rules api/site-config	1 hora
P1 (Alta) Prazo: 24h	C5-01	Substituir sanitização regex por isomorphic-dompurify no blog	src/lib/security.ts	1 hora
P1 (Alta) Prazo: 24h	C5-02	Adicionar regex alfanumérica em gald e gtmdl antes de interpolar em scripts inline	src/app/layout.tsx	30 min
P2 (Média) Sprint Atual	C2-01	Emitir cookie HttpOnly admin_session no fluxo de fallback de login via Firebase Auth	src/services/authService.ts	2 horas
P2 (Média) Sprint Atual	C4-01	Remover lista de e-mails administrativos do bundle client-side	src/services/authService.ts	1 hora
P2 (Média) Sprint Atual	C4-02	Substituir comparação === por timingSafeEqual no endpoint de login	api/admin/login/route.ts	30 min
P3 (Baixa) Próxima Sprint	C2-02	Adicionar guard de middleware para rotas /api/admin/* (Defesa em Profundidade)	src/middleware.ts	1 hora
P3 (Baixa) Próxima Sprint	C3-01	Padronizar validação de formato de IDs dinâmicos em posts e cursos	api/posts/[id] api/courses/[id]	45 min
P3 (Baixa) Próxima Sprint	C4-03	Garantir rotação de credenciais de produção e saneamento de histórico Git	Repositório Git	1 hora

5. Modelos de GitHub Issues Prontos para Abertura

Os modelos abaixo estão formatados para cópia direta no GitHub Issues com passos de reprodução e correção:

```
--- ISSUE 1 ---
**Título:** security: ocultar n8nWebhookUrl na API pública e restringir leitura no Firestore
**Labels:** `security, backend, high-priority`

#### Descrição
A URL privada do webhook n8n é exposta no endpoint público GET /api/site-config e pode ser lida por clientes não autenticados via Firestore SDK devido à regra aberta match /config/{configId} { allow read: if true; }.

#### Passos para Reproduzir
1. Executar `curl -s https://www.easytraining.com.br/api/site-config`
2. Observar o campo `n8nWebhookUrl` retornado no JSON público.

#### Correção Proposta
1. Em `src/app/api/site-config/route.ts`, filtrar `n8nWebhookUrl` da resposta pública.
2. Em `firestore.rules`, alterar a regra de `/config/{configId}` para `allow read, write: if false;`.
--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
**Título:** security: substituir sanitizador HTML por DOMPurify para prevenir Stored XSS no Blog
**Labels:** `security, frontend, high-priority`

#### Descrição
A função `sanitizeHtmlContent` em `src/lib/security.ts` utiliza regex ingênua que pode ser contornada por tags maliciosas ou manipuladores sem espaçamento padrão (ex: `
```

```
--- ISSUE 3 ---
**Título:** security: validar formato de IDs de GA4 e GTM no RootLayout
**Labels:** `security, frontend, high-priority`

#### Descrição
No arquivo `src/app/layout.tsx`, `googleAnalyticsId` e `googleTagManagerId` são interpolados diretamente em scripts inline. Se um valor contendo aspas for gravado nas configurações, ele causará Stored XSS generalizado.

#### Passos para Reproduzir
1. Atualizar siteConfig com `googleAnalyticsId: "G-TEST"; alert(1); //"`
2. Acessar qualquer página do site e verificar a quebra e execução indevida de JavaScript.

#### Correção Proposta
Validar com `/^[A-Z0-9-]{4,20}$/i` antes de injetar os identificadores no template literal do script.
--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---
**Título:** security: sincronizar sessão server-side no fallback de login do Firebase Auth
**Labels:** `security, auth, medium-priority`

#### Descrição
Quando o login é realizado pelo fallback do Firebase Auth no cliente, apenas o `localStorage` é preenchido. O cookie HttpOnly `admin_session` não é gerado, resultando em interface administrativa aberta no navegador mas com todas as requisições de API bloqueadas com 401.

#### Passos para Reproduzir
1. Efetuar login através do fallback Firebase Auth.
2. O dashboard abre, mas ao tentar salvar qualquer post ou curso, a requisição é rejeitada pelo backend.

#### Correção Proposta
Fazer com que o fluxo envie o `idToken` do Firebase para o endpoint `/api/admin/login` e receba o cookie `admin_session` oficial.
--- FIM ISSUE 4 ---
```

--- ISSUE 5 ---

****Título:**** security: remover ALLOWED_ADMIN_EMAILS do bundle público do cliente
****Labels:**** `security, hardening, medium-priority`

Descrição

O array `ALLOWED\_ADMIN\_EMAILS` está no arquivo `src/services/authService.ts`, que é incluído no bundle público gerado pelo Webpack/Turbopack, expondo a lista de e-mails de todos os administradores.

Passos para Reproduzir

1. Inspecionar o código-fonte empacotado no navegador.
2. Buscar por `@easytraining.com.br` e visualizar os e-mails administrativos.

Correção Proposta

Manter a lista de e-mails estritamente no backend (`src/lib/authServer.ts`).

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

****Título:**** security: utilizar timingSafeEqual na validação de senha mestra
****Labels:**** `security, backend, medium-priority`

Descrição

A verificação de senha administrativa no endpoint `POST /api/admin/login` utiliza o operador `===`, suscetível a ataques de tempo (timing attacks) para adivinhação da senha caractere por caractere.

Passos para Reproduzir

Análise estática do arquivo `src/app/api/admin/login/route.ts` linha 37.

Correção Proposta

Utilizar `crypto.timingSafeEqual` com buffers de comprimento padronizado.

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

****Título:**** security: adicionar guard de autenticação para /api/admin/* no middleware
****Labels:**** `security, architecture, low-priority`

Descrição

O middleware atual em `src/middleware.ts` isenta todas as rotas `/api`. Embora cada rota verifique individualmente o token, a inclusão de um guard centralizado adiciona a camada necessária de Defesa em Profundidade.

Correção Proposta

Adicionar verificação de presença e formato do cookie `admin\_session` para requisições em `/api/admin/\*` diretamente no middleware.

--- FIM ISSUE 7 ---